SIMATS ENGINEERING

ASSESSMENT TOOL 3

DEBUGGING

CO2 – Query Processing for Data Science

Student Name	Answar Vijes D
Register Number	192324028
Course Name	Query Processing for Data Science
Course Code	DSA0501
Assessment	Assessment Tool 3 – Debugging
Course Outcome	CO2

Complete answers with error identification, explanations, corrected programs, and expected-result notes.

General Notes

The programs below follow the database types and question wording in the supplied assessment. For MySQL and PostgreSQL, the database server, database, tables, credentials, and required Python packages must already be configured. The sample outputs are illustrative; actual retrieved rows and affected-row counts depend on the database contents.

Required packages (install only the drivers you need): pip install mysql-connector-python psycopg2-binary. SQLite support is included with Python through sqlite3.

Question 1: SQLite Database Connection

Errors Identified and Explanation

1. The database filename student.db is not enclosed in quotation marks. Python therefore treats it as a variable instead of a filename string.

2. conn.cursor is not called. It refers to the cursor method rather than creating a cursor object.

3. conn.commit is not called. The transaction is not explicitly committed.

Corrected Python Program

```
import sqlite3

conn = sqlite3.connect("student.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE Student (
    Student_ID INTEGER PRIMARY KEY,
    Name TEXT,
    Department TEXT
)
""")

conn.commit()
cursor.close()
conn.close()

print("Student table created successfully.")
```

Expected Output

```
Student table created successfully.
```

Note: If the Student table already exists, CREATE TABLE will raise an error. Run this once, or use CREATE TABLE IF NOT EXISTS if repeated execution is required.

Question 2: MySQL Record Insertion

Errors Identified and Explanation

1. The INSERT statement has three placeholders (%s, %s, %s), but only two values are supplied.

2. The values must be passed together as a tuple (or another supported sequence) containing Student_ID, Name, and Department.

Corrected Python Program

```
import mysql.connector

conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="root",
    database="college"
)
```

```
cursor = conn.cursor()

query = """
INSERT INTO Student (Student_ID, Name, Department)
VALUES (%s, %s, %s)
"""

values = (101, "Alice", "CSE")
cursor.execute(query, values)

conn.commit()
print("Student record inserted successfully.")

cursor.close()
conn.close()
```

Expected Output

```
Student record inserted successfully.
```

Note: The example uses CSE as the Department value to complete the three-column record. The Student table must contain the specified columns, and Student_ID 101 must not already exist.

Question 3: PostgreSQL Data Retrieval

Errors Identified and Explanation

1. fetchone() retrieves only one row, but the program then tries to iterate through rows as if it were a collection of all records.

2. To retrieve every record, use fetchall().

3. The cursor and connection should both be closed after retrieval.

Corrected Python Program

```
import psycopg2

conn = psycopg2.connect(
    host="localhost",
    database="company",
    user="postgres",
    password="admin"
)

cursor = conn.cursor()
cursor.execute("SELECT * FROM Employee")

rows = cursor.fetchall()

for row in rows:
    print(row)

cursor.close()
conn.close()
```

Expected Output

```
(101, 'Alice', 50000)
(102, 'Bob', 60000)
```

Note: These rows are examples only. The actual output contains all rows currently stored in the Employee table; if the table is empty, nothing is printed.

Question 4: SQLite Record Update

Errors Identified and Explanation

1. The SQL statement contains one placeholder (?) for Student_ID, but cursor.execute() is called without supplying its value.

2. SQLite parameters must be passed as a sequence. For a single value, use a one-element tuple such as (105,).

Corrected Python Program

```
import sqlite3

conn = sqlite3.connect("college.db")
cursor = conn.cursor()

query = """
UPDATE Student
SET Department = 'ECE'
WHERE Student_ID = ?
"""

student_id = 105
cursor.execute(query, (student_id,))

conn.commit()
print("Student department updated.")

cursor.close()
conn.close()
```

Expected Output

```
Student department updated.
```

Note: The message confirms that the command was executed, not necessarily that a row matched. To check whether a row was changed, inspect cursor.rowcount before closing the cursor.

Question 5: PostgreSQL Record Deletion

Errors Identified and Explanation

1. The DELETE statement contains a %s placeholder but the customer_id value is not passed to execute().

2. The deletion should be committed using conn.commit().

3. The cursor should be closed as well as the connection.

Corrected Python Program

```
import psycopg2

conn = psycopg2.connect(
    host="localhost",
    database="bank",
    user="postgres",
    password="admin"
)

cursor = conn.cursor()

customer_id = 105
query = "DELETE FROM Customer WHERE Customer_ID = %s"

cursor.execute(query, (customer_id,))
conn.commit()

print("Customer record deletion command executed.")

cursor.close()
conn.close()
```

Expected Output

```
Customer record deletion command executed.
```

Note: The message does not prove that a customer was found. The actual number of deleted rows depends on whether Customer_ID 105 exists.

Question 6: SQLite Record Retrieval

Errors Identified and Explanation

1. student_id is passed as a single integer. The SQLite execute() method expects the query parameters as a sequence, so use (student_id,).

2. fetchall() returns a list of rows, and row[0] can fail if no matching record exists. fetchone() is simpler when only one student is requested.

3. The program should check whether a matching row was returned before printing it.

Corrected Python Program

```
import sqlite3

conn = sqlite3.connect("college.db")
cursor = conn.cursor()

student_id = 105

cursor.execute(
    "SELECT * FROM Student WHERE Student_ID = ?",
    (student_id,)
)

row = cursor.fetchone()

if row is not None:
    print(row)
else:
    print("No student found with Student_ID 105.")

cursor.close()
conn.close()
```

Expected Output

```
(105, 'Alice', 'ECE')
```

Note: The displayed row is an example only. If Student_ID 105 is absent, the program prints the 'No student found' message instead.

Question 7: MySQL Table Creation

Errors Identified and Explanation

1. A comma is missing after Student_ID INT PRIMARY KEY. SQL requires commas between column definitions.

2. The table creation statement can otherwise use the supplied column definitions.

Corrected Python Program

```
import mysql.connector

conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="root",
    database="college"
)

cursor = conn.cursor()

cursor.execute("""
CREATE TABLE Student (
    Student_ID INT PRIMARY KEY,
    Name VARCHAR(50),
    Department VARCHAR(30)
```

```
)
""")

conn.commit()
print("Student table created successfully.")

cursor.close()
conn.close()
```

Expected Output

```
Student table created successfully.
```

Note: If the table already exists, CREATE TABLE will fail. For safe repeated execution, use CREATE TABLE IF NOT EXISTS Student (...).

Question 8: PostgreSQL Record Insertion

Errors Identified and Explanation

1. cursor.execute() is given the SQL statement and three separate values as positional arguments. psycopg2 expects the parameter values together in one sequence.

2. The values should be supplied as a tuple: (101, 'John', 55000).

Corrected Python Program

```
import psycopg2

conn = psycopg2.connect(
    host="localhost",
    database="company",
    user="postgres",
    password="admin"
)

cursor = conn.cursor()

sql = """
INSERT INTO Employee (Employee_ID, Name, Salary)
VALUES (%s, %s, %s)
"""

values = (101, "John", 55000)
cursor.execute(sql, values)

conn.commit()
print("Employee record inserted successfully.")

cursor.close()
conn.close()
```

Expected Output

```
Employee record inserted successfully.
```

Note: The Employee table must have the specified columns and compatible data types. Employee_ID 101 must not already exist if it is a primary key.

Question 9: MySQL Record Update

Errors Identified and Explanation

1. cursor.execute() is called with salary and emp_id as separate arguments. MySQL Connector expects the parameter values together in a tuple or sequence.

2. The correct parameter order is (salary, emp_id), matching the order of the placeholders in the SQL statement.

Corrected Python Program

```
import mysql.connector

conn = mysql.connector.connect(
    host="localhost",
    user="root",
    password="root",
    database="company"
)

cursor = conn.cursor()

salary = 65000
emp_id = 110

query = """
UPDATE Employee
SET Salary = %s
WHERE Employee_ID = %s
"""

cursor.execute(query, (salary, emp_id))
conn.commit()

print("Update command executed.")
print("Rows affected:", cursor.rowcount)

cursor.close()
conn.close()
```

Expected Output

```
Update command executed.
Rows affected: 1
```

Note: Rows affected is an example. It may be 0 if Employee_ID 110 does not match a row, or may vary depending on the database and whether the value changes.

Question 10: PostgreSQL Record Deletion

Errors Identified and Explanation

1. The DELETE statement contains a %s placeholder for City, but cursor.execute() is called without passing the city value.

2. The city value must be passed as a one-element tuple: (city,).

3. The transaction should be committed after executing the DELETE statement so the deletion is saved.

Corrected Python Program

```
import psycopg2

conn = psycopg2.connect(
    host="localhost",
    database="bank",
    user="postgres",
    password="admin"
)

cursor = conn.cursor()

city = "Chennai"
query = "DELETE FROM Customer WHERE City = %s"

cursor.execute(query, (city,))
conn.commit()

print("Delete command executed.")
print("Rows deleted:", cursor.rowcount)

cursor.close()
conn.close()
```

Expected Output

```
Delete command executed.
Rows deleted: 3
```

Note: The number 3 is only an illustration. The actual number of deleted records depends on how many Customer rows have City equal to Chennai.

Final Verification Checklist

• All 10 questions from the supplied assessment are answered in the original order.

• Each question includes identified errors, short explanations, corrected code, and an expected-result note.

• SQLite uses ? placeholders; MySQL Connector and psycopg2 use %s placeholders.

• Parameter values are passed as tuples or sequences, including one-element tuples where required.

• Database changes are committed, and cursors and connections are closed.

• Illustrative outputs are clearly distinguished from actual database-dependent results.

End of Assessment Tool 3 – Debugging Answers